

Codebase Overview

Repository inventory, architecture, testing, coverage, dependencies, and maintainability snapshot

Prepared July 18, 2026

382 PROJECT FILES	332 CODE FILES	42,366 CODE LINES
1.44 MiB CODE FOOTPRINT	403 PASSING TESTS	Clean GIT WORKTREE

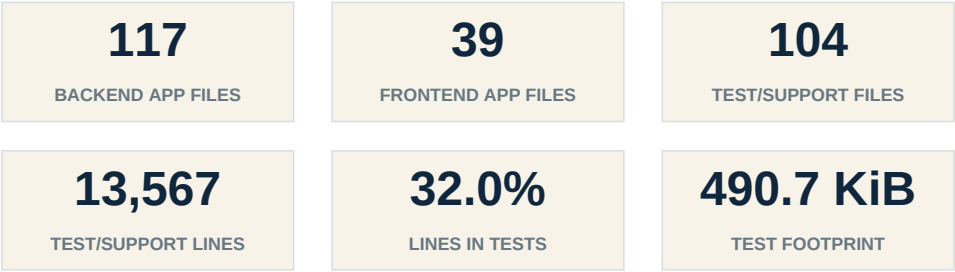
DiamondIQ is a local-first baseball intelligence application. A Rails JSON API and PostgreSQL data model ingest MLB Stats API and Baseball Savant data, produce versioned analytics and benchmarks, and serve a Vue 3 application for home, Stat Explorer, player, team, game, and administration workflows.

STACK	DATA FLOW
Ruby 3.2.3 / Rails 7.1.6 / PostgreSQL / RSpec 6.1.5 Vue 3.5.35 / Vite 8.0.16 / Vitest 4.1.8	MLB Stats API + Baseball Savant -> synchronization/import services -> PostgreSQL -> daily analytics and benchmarks -> Rails API -> Vue application

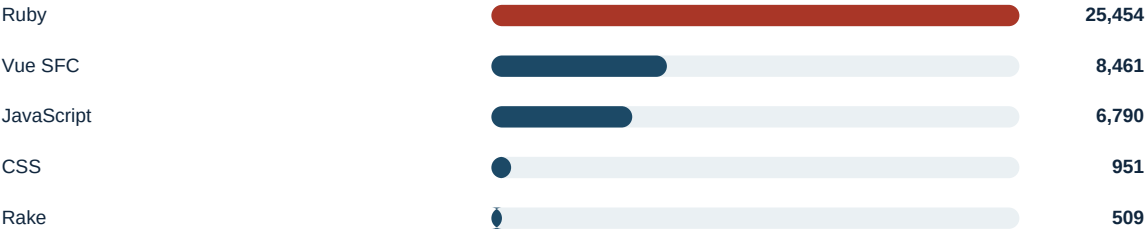
REPOSITORY PROFILE

Inventory and composition

The source-oriented inventory excludes generated dependencies, Git internals, runtime logs, temporary files, downloaded datasets, and compiled frontend output. It contains 382 project files totaling 3.63 MiB; 332 are classified as code.



CODE LINES BY MAJOR TYPE



ARCHITECTURE COUNTS

Rails models	31	Rails controllers	14	Services	46
Queries	13	Serializers	3	Jobs	7
Vue views	6	Vue components	11	Composables	16
Database tables	40	Migrations	33	Rake tasks	10

CONCENTRATION AND MAINTAINABILITY

Largest code files

File size is not a quality judgment, but large mixed-responsibility files increase navigation cost, review surface, merge conflicts, and regression risk. The list below ranks code files by bytes and includes line counts for context.

#	File	Size	Lines	Area
1	frontend/src/views/AdminView.vue	97.6 KiB	3,212	Frontend application
2	frontend/src/views/GameSummaryView.vue	58.2 KiB	919	Frontend application
3	backend/db/schema.rb	44.5 KiB	949	Database
4	frontend/src/views/PlayerProfileView.vue	34.5 KiB	1,263	Frontend application
5	frontend/src/views/__tests__/AdminView.spec.js	31.7 KiB	756	Tests
6	frontend/src/views/TeamProfileView.vue	30.6 KiB	505	Frontend application
7	frontend/src/views/__tests__/GameSummaryView.spec.js	29.5 KiB	630	Tests
8	backend/app/queries/team_profile_snapshot_query.rb	27.4 KiB	860	Backend application
9	frontend/src/components/PlayerSeasonStatsDashboard.vue	26.7 KiB	816	Frontend application
10	backend/app/queries/player_profile_snapshot_query.rb	24.5 KiB	725	Backend application

CONCENTRATION	PRIMARY HOTSPOTS
The ten largest files contain 405.2 KiB, or 27.4% of the code footprint.	AdminView.vue is the clearest decomposition candidate at 3,212 lines. PlayerProfileView.vue and GameSummaryView.vue are also feature-dense. The largest backend query objects centralize profile aggregation logic and merit targeted contract tests.

SUGGESTED DECOMPOSITION ORDER

1. Extract Admin task cards, database dialogs, and data-health panels into focused Vue components.
2. Split Game Summary analytical tabs into independently tested tab components.
3. Move Player Profile sections and formatting logic behind section components and composables.
4. Preserve query-object API contracts while extracting reusable stat aggregation helpers.

QUALITY SIGNAL

Test suite and measured coverage

309 RAILS EXAMPLES PASSED	94 FRONTEND TESTS PASSED	403 TOTAL PASSING TESTS
93.48% BACKEND LINE COVERAGE	90.74% FRONTEND LINE COVERAGE	73.19% FRONTEND BRANCH COVERAGE

COVERAGE SUMMARY

Suite	Metric	Covered / total	Coverage	Method
Rails backend	Lines	5,366 / 5,740	93.48%	Ruby built-in Coverage
Vue frontend	Lines	1,853 / 2,042	90.74%	Vitest V8
Vue frontend	Statements	2,022 / 2,363	85.56%	Vitest V8
Vue frontend	Functions	601 / 695	86.47%	Vitest V8
Vue frontend	Branches	2,427 / 3,316	73.19%	Vitest V8

INTERPRETATION

Both suites passed with zero failures and zero pending Rails examples. Line coverage is high in both applications. Frontend branch coverage is the clearest aggregate gap: 889 of 3,316 measured branches were not exercised. Coverage is a useful risk indicator, not proof of behavioral correctness; data-import boundaries and failure paths deserve disproportionate attention.

TEST INVENTORY

The repository contains 75 RSpec specification files and 20 Vitest specification files. The broader 104-file test/support inventory also includes Rails test helpers and support code. Test/support code accounts for 13,567 lines, or 32.0% of all classified code lines.

WHERE ADDITIONAL TESTS PAY OFF

Coverage gaps and test priorities

LOWEST BACKEND LINE COVERAGE (20+ EXECUTABLE LINES)

File	Covered / total	Coverage
backend/app/services/mlb_game_details_downloader.rb	24 / 38	63.16%
backend/app/services/mlb_player_profiles_downloader.rb	29 / 43	67.44%
backend/app/services/mlb_player_transactions_downloader.rb	34 / 49	69.39%
backend/app/services/mlb_roster_downloader.rb	37 / 52	71.15%
backend/app/services/mlb_schedule_downloader.rb	39 / 54	72.22%
backend/app/services/pitch_data_importer.rb	104 / 141	73.76%
backend/app/services/player_stats_downloader.rb	94 / 113	83.19%
backend/app/services/pitch_data_downloader.rb	85 / 102	83.33%

LOWEST FRONTEND LINE COVERAGE (20+ EXECUTABLE LINES)

File	Lines	Branches	Functions
src/components/PlayerSeasonStatsDashboard.vue	83.78%	80.85%	74.39%
src/composables/useGameDetailsSync.js	84.88%	53.84%	83.33%
src/components/CsvImportPicker.vue	85.50%	69.84%	86.66%
src/views/AdminView.vue	86.95%	77.63%	74.59%
src/components/PlayerSearch.vue	87.34%	89.06%	62.50%
src/composables/usePlayerSeasonStatsImport.js	87.50%	74.07%	100.00%
src/composables/usePlayerSuggestions.js	88.23%	58.62%	100.00%
src/composables/useTeamProfile.js	88.57%	52.05%	88.88%

Priority: add timeout, malformed-response, retry, partial-import, and cancellation scenarios around downloader/importer services. On the frontend, emphasize conditional paths in game synchronization, Admin operations, player suggestions, and team-profile loading. These areas have meaningful branching and external-state behavior.

OPERATIONAL PROFILE

Disk footprint, dependencies, and next actions

Area	Disk size	Notes
Project files (source-oriented)	3.63 MiB	Excludes dependencies, Git, datasets, logs, temp, and builds
Classified code	1.44 MiB	332 code files
Frontend node_modules	110.66 MiB	Installed development/runtime dependencies
Backend vendor	110.88 MiB	Vendored Ruby bundle footprint
Git metadata	61.29 MiB	Repository history and objects

DEPENDENCY PROFILE

Stack	Direct runtime	Locked/dev	Core packages
Rails backend	10	99	Rails, PostgreSQL, Puma, Solid Queue, RSpec, SeedFu
Vue frontend	2	6	Vue, Vue Router, Vite, Vitest, Vue Test Utils, jsdom

RECOMMENDED NEXT ACTIONS

1	Persist coverage in CI	Add SimpleCov or an equivalent Rails coverage setup and @vitepress/coverage-v8 as a dev dependency. Publish artifacts and enforce gradual thresholds rather than a sudden hard gate.
2	Reduce frontend concentration	Decompose AdminView first, then GameSummaryView and PlayerProfileView. Preserve tab- and section-level tests while moving presentation into focused components.
3	Harden external data boundaries	Raise coverage for MLB and Baseball Savant downloaders, pitch imports, and batch synchronization, especially transient failures and partial payloads.
4	Update documentation	The root README still describes the old home route as the Stat Board. It should document Home, /explore Stat Explorer, /games/:id, and the expanded Game Summary analytics.
5	Track structural trends	Record code lines, largest files, coverage, migration count, and dependency footprint in CI so growth and concentration changes are visible over time.

METHODOLOGY AND LIMITATIONS

Inventory was measured from the working tree on branch main at commit 2ebfe61d with a clean status. Code classification includes Ruby, Rake, Vue, JavaScript, CSS, SQL, shell, and executable project scripts. It excludes .git, node_modules, vendor, data, dist, output, logs, storage, temporary files, and editor metadata. Backend line coverage was measured with Ruby's built-in Coverage API while running the full RSpec suite; branch and method coverage were not measured. Frontend coverage used Vitest's V8 provider, installed temporarily without changing package manifests. Coverage results represent this test run and may differ with environment or test selection.